

Underwater Pipeline Defect Detection: A Four-Architecture Comparison

**Interim Report on
Deep Learning Project
[ICT-4442]**

Submitted By

**Krishna Anand, 230953398
Harish Bharathwaj, 230953594
Siddhartha Saha, 230911150
Pushkar Ojha, 230911156**



MANIPAL
ACADEMY *of* HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

**SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION**

OCTOBER 2026

INTERIM REPORT (Phase 2)

Team Member Names (with Registration Numbers):

Krishna Anand (230953398), Harish Bharathwaj (230953594), Siddhartha Saha (230911150), Pushkar Ojha (230911154).

Title of the Project:

Underwater Pipeline Defect Detection: A Four-Architecture Comparison

GitHub Repository Link:

[krishna-086/underwater-pipeline-defects](https://github.com/krishna-086/underwater-pipeline-defects)

1. Literature Review

Paper (Author, Year)	Method	Dataset	Key Result	Relevance to Project
[1] Ren, He, Girshick, Sun (2015), Faster R-CNN	Two-stage detector. A Region Proposal Network shares convolutional features with the detection head, so proposals cost almost nothing.	PASCAL VOC 2007/2012, MS COCO	73.2% mAP on VOC07 with VGG-16 at roughly 5 fps on a GPU. Proposal time fell from seconds to 10 ms.	Our two-stage model. Sets the accuracy bar the one-stage models get measured against.
[2] Lin, Dollar, Girshick, He, Hariharan, Belongie (2017), FPN	Feature Pyramid Network. A top-down pathway with lateral connections produces good features at every scale from one backbone.	MS COCO	Faster R-CNN with FPN hit 36.2 AP on COCO test-dev, the best single-model number at the time.	The neck under both Faster R-CNN and RetinaNet here. Multi-scale features are our best hope for thin cracks.
[3] Lin, Goyal, Girshick, He, Dollar (2017), RetinaNet	One-stage anchor-based detector. Focal loss scales down the loss on easy negatives so the dense head is not drowned by background.	MS COCO	39.1 AP on COCO test-dev. First one-stage detector to match two-stage accuracy.	Our RetinaNet. Same backbone as Faster R-CNN, so whatever gap appears between them is the one-stage vs two-stage gap.
[4] Redmon, Divvala, Girshick, Farhadi (2016), YOLO	One network regresses boxes and classes from a grid in a single pass. No proposal stage.	PASCAL VOC 2007/2012	63.4 mAP on VOC07 at 45 fps, Fast YOLO at 155 fps. Less accurate than Faster R-CNN, much faster.	Where the one-stage family starts. It frames the speed against accuracy trade we are measuring.
[5] Jocher, Chaurasia, Qiu (2023), YOLOv8	Anchor-free one-stage detector. C2f backbone, decoupled head, CIoU plus distribution focal loss.	MS COCO	YOLOv8n: 37.3 mAP50-95 on COCO val with 3.2M parameters. YOLOv8x: 53.9.	Our YOLOv8n. Smallest model in the slate, defines the speed end. Anchor-free should suit long, thin crack boxes.
[6] Carion, Massa, Synnaeve, Usunier, Kirillov, Zagoruyko (2020), DETR	Transformer encoder-decoder that predicts a set of boxes directly. Hungarian matching replaces anchors and NMS.	MS COCO	42.0 AP with ResNet-50, level with Faster R-CNN, but took 500 epochs and struggled on small objects.	Where the attention family starts. Its slow convergence is the exact risk with 862 training images.
[7] Zhu, Su, Lu, Li, Wang, Dai (2021), Deformable DETR	Deformable attention looks at a handful of sampled points per query across multi-scale features instead of the whole image.	MS COCO	43.8 AP in 50 epochs, a tenth of what DETR needed, with better small-object AP.	Explains how a DETR descendant can train on little data. Direct ancestor of our RT-DETR.
[8] Zhao, Lv, Xu, Wei, Wang, Dang, Liu, Chen (2024), RT-DETR	Hybrid encoder separates within-scale attention from cross-scale fusion. Uncertainty-minimal query selection. No NMS.	MS COCO	RT-DETR-L: 53.0 AP at 114 fps on a T4. RT-DETR-R50: 53.1 AP at 108 fps. Ahead of YOLOv8-L on both.	Our RT-DETR-L. The first end-to-end detector that runs in real time, and the transformer entry in our comparison.
[9] Fu, Liu, Fan, Chen, Fu, Yuan, Zhu, Luo (2023), RUOD	Builds a 14,000-image underwater benchmark and tests generic detectors (RetinaNet, Cascade R-CNN, ATSS and others) against underwater-specific ones.	RUOD, 10 classes, 74,903 objects	Generic detectors lose accuracy under colour distortion and uneven lighting. Methods with edge enhancement and decolourisation held up better.	Direct evidence that COCO rankings shift once you go underwater. Our frames have the same colour cast and lighting problems.
[10] Cheng, Wang (2018), sewer pipe defects	Faster R-CNN on CCTV pipe-inspection frames, with tuning of filter size and stride.	3,000 CCTV sewer images	83% mAP after hyperparameter tuning. Authors report high accuracy and fast speed.	Closest published task to ours, defects on pipe interiors. One detector only, no comparison across families.

Taken together, the papers trace a fairly clean arc. Faster R-CNN [1] set the accuracy bar by spending compute on proposals, and FPN [2] gave it decent features at every scale. RetinaNet [3] then showed the one-stage accuracy gap was a loss problem rather than an architecture problem, and closed it with focal loss. YOLO [4] went the other way, trading a few mAP points for an order of magnitude in speed, and YOLOv8 [5] kept pushing by dropping anchors. DETR [6] threw out the whole box-and-NMS machinery in favour of set prediction. That was tidy on paper but took 500 epochs; Deformable DETR [7] brought it down to 50, and RT-DETR [8] got it to real time, where it now beats same-size YOLO models on COCO.

What none of them settle is which family wins on our data. Every COCO figure above comes from 118,000 training images of ordinary objects photographed in air. RUOD [9] shows generic detectors slip once water distorts colour and light. Cheng and Wang [10] reached 83% mAP on pipe defects but only tried Faster R-CNN. Nobody has lined the four families up on underwater pipeline defects with under a thousand training images. That is the hole this project is meant to fill.

2. Dataset Acquisition and Preprocessing

Source. We use the COROSSION DETECTION project on Roboflow Universe [11], licensed CC BY 4.0. Harish assembled it by merging several public Roboflow datasets into one project with a single class list, crack and corrosion. It contains 1,214 source images with bounding box labels. The annotations are polygons. We export in YOLOv8 format, which carries the polygon outlines through, and convert each one to an axis-aligned box from its minimum and maximum extent (data/poly2bbox.py). A few exported lines had two coordinates run together with no separator; the same script repairs those before conversion.

Resize and augmentation. Roboflow resizes each image to fit inside 640 x 640 before export. Augmentation runs on the training split only, three outputs per source image: hue shift of plus or minus 15 degrees, brightness of plus or minus 25 percent, Gaussian blur up to 2.6 px. We chose those three because they imitate what changes between underwater frames (colour cast, exposure, turbidity) without touching box geometry. There is no geometric augmentation at export. Ultralytics adds horizontal flip and mosaic during training; the torchvision loop adds horizontal flip only.

Split. The export carries its own split. We froze it rather than reshuffling, so every model trains, validates and tests on exactly the same images:

Split	Source images	After augmentation	Share of source
Train	862	2,586	71.0%
Validation	194	194	16.0%
Test	158	158	13.0%

Pipeline steps, in order. Each one is a script under data/ in the repository.

0. poly2bbox.py. Converts the exported polygon outlines to normalised centre-x, centre-y, width, height boxes in place, keeping the originals in labels_poly/. Ultralytics would do this conversion internally on its own, but the torchvision path and the counting script need plain boxes on disk

1. dedup.py. The corpus was merged from several sources, so the same photo can turn up twice under different names, and a random split has no way of knowing. We compute a 64-bit perceptual hash (pHash) for every image in all three splits and group images whose hashes sit within Hamming distance 6. A group that straddles two splits is moved wholesale into train, so nothing in validation or test has a near-twin the model has seen. The script prints how many groups it found and how many files it moved.

2. freeze_split.py. Checks that every image has a label file, every class id is 0 or 1, and every box coordinate lies in [0, 1]. Then it writes splits/manifest.json with the exact filenames per split. All four members train from this manifest. It also prints the per-class counts below, so class imbalance is on the page rather than buried in an aggregate mAP.

Class	Train instances	Val instances	Test instances
crack	2307	30	52
corrosion	3384	480	326

3. yolo2coco.py. torchvision's Faster R-CNN and RetinaNet want absolute xxyy boxes, and pycocotools reads COCO JSON, so the YOLO labels are converted once per split. Class ids shift up by one because torchvision keeps 0 for background.

Normalisation. Ultralytics scales pixels to [0, 1] and letterboxes on its own. The torchvision path applies ImageNet mean and standard deviation, since its ResNet-50 backbone was pretrained on ImageNet. Both paths see 640 px input.

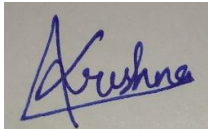
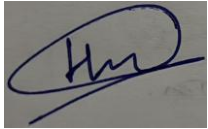
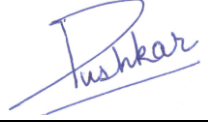
Evaluation. Each framework reports its own mAP, and they interpolate the PR curve slightly differently. For the final comparison every model's test predictions are exported to COCO JSON and scored once by pycocotools. The interim numbers below are each framework's own validation mAP, which is fine for a first look and not for the final table.

3. Models Implemented So Far

Model ownership was adjusted after the synopsis. The two Ultralytics models, which share one training interface, are now with Krishna and Harish so that both preliminary runs land before this report; the two torchvision models, which share one custom training loop, are with Pushkar and Siddhartha.

Model	Owner (Member)	Status	Preliminary Metric (val split)	Notes
YOLOv8n	Krishna Anand	Training script committed; 20-epoch preliminary run complete	mAP@0.5: 0.528 mAP@0.5:0.95: 0.304 Precision: 0.572 Recall: 0.485 (epoch 20, from results.csv)	Ultralytics 8.3, yolov8n.pt COCO weights, imgsz 640, batch 16, SGD defaults, seed 0. Published ceiling on COCO val: 37.3 mAP50-95 [5].
RT-DETR-L	Harish Bharathwaj	Training script committed; 20-epoch preliminary run complete	mAP@0.5: 0.384 mAP@0.5:0.95: 0.206 Precision: 0.441 Recall: 0.352 (epoch 20, from results.csv)	Ultralytics rtdetr-l.pt, imgsz 640, batch 8 for memory, AdamW lr 1e-4. Published COCO val: 53.0 AP [8]. Likely to trail the CNNs at this data size, and that gap is itself a finding.
Faster R-CNN (ResNet-50 FPN)	Pushkar Ojha	Model definition committed. Shared torchvision loop in progress.	Pending	fasterrcnn_resnet50_fpn_v2 with COCO weights; box predictor replaced with 3 outputs (2 classes plus background).
RetinaNet (ResNet-50 FPN)	Siddhartha Saha	Model definition and loop skeleton committed. Pending.	Pending	retinanet_resnet50_fpn_v2 with COCO weights; classification head rebuilt for 2 classes.

4. Individual Contribution Log (to date)

Member Name	Reg. No.	Task(s) Completed	Signature
Krishna Anand	230953398	Synopsis drafting and submission. Repository setup and structure. freeze_split.py (label integrity check, frozen manifest, per-class counts). YOLOv8n training script and preliminary run. Common evaluation protocol (metrics, pycocotools scoring) for all four models.	
Harish Bharathwaj	230953594	Dataset consolidation from multiple Roboflow sources and publication on Roboflow Universe. dedup.py perceptual-hash pass. yolo2coco.py conversion for the torchvision models. RT-DETR-L training script and preliminary run.	
Siddhartha Saha	230911150	Literature review: selection and reading of the ten papers, the summary table and the synthesis in Section 1. torchvision Dataset class and shared training loop skeleton used by both torchvision models. RetinaNet model definition.	
Pushkar Ojha	230911156	Faster R-CNN model definition and head replacement. Latency and throughput benchmark script (same hardware, batch and input for all four models). Compilation and formatting of this interim report.	

Every member commits their own files from their own GitHub account. The repository is laid out as data/, train/, eval/ and report/, with one branch per member merged to main through pull requests, so the commit history lines up with the rows above.

5. Risk / Plan for Remaining Work

Remaining models. Faster R-CNN and RetinaNet have not trained yet. Both sit on one shared torchvision loop (Siddhartha), with Pushkar and Siddhartha each owning one model definition and its tuning. Once the loop runs end to end for one of them, the second is a model swap.

Expected challenges and how we handle them:

1. The torchvision loop is the single biggest risk. Ultralytics hands you a training loop, a collate function and mAP; torchvision hands you none of them. We follow the torchvision detection reference (engine.py pattern) and score with pycocotools. Two weeks are set aside and work starts now.
2. GPU memory. Faster R-CNN R50-FPN at 640 fits an 8 GB card only at batch 4 or lower. We use mixed precision and batch 4, with a Colab or Kaggle T4 as fallback if the local card runs out.

3. RT-DETR on 862 images. The DETR family is data-hungry, and RT-DETR-L has about 32M parameters against YOLOv8n's 3.2M. It will probably trail the CNNs. We give it a longer schedule (50 to 100 epochs) and write the gap up as a result about data efficiency, which is objective 3 from the synopsis.
4. The test split is 158 images. Two models a point or two of mAP apart cannot be called different. We bootstrap the test set (resample images with replacement, 1,000 draws) to attach a confidence interval to every number in the final table, and say so plainly when a gap sits inside the noise.
5. Metric mismatch across frameworks. Ultralytics and pycocotools interpolate the PR curve differently and can disagree by a point or two. The final table uses pycocotools on exported predictions for all four models. A framework's own number is never compared across models.
6. De-duplication may shrink the usable set if many cross-split duplicates turn up. We report before and after counts and retrain the two Ultralytics models on the cleaned manifest so every model sees identical data.

Timeline to Final submission:

Milestone	Owner	Date
Interim report submitted; YOLOv8n and RT-DETR-L preliminary runs committed	All	2 Oct 2026
torchvision loop running end to end on one model	Siddhartha	9 Oct 2026
Faster R-CNN and RetinaNet preliminary results	Pushkar, Siddhartha	14 Oct 2026
Final training runs, all four, matched protocol on cleaned manifest	All	20 Oct 2026
pycocotools comparison table, bootstrap CIs, per-class error analysis	Krishna	23 Oct 2026
Latency benchmark on one machine; FastAPI demo with best model	Pushkar, Harish	26 Oct 2026
Final report, Turnitin check, slides	All	29 Oct 2026
Presentation and submission with repository	All	31 Oct 2026

References

- [1] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2015, pp. 91–99.
- [2] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: <https://docs.ultralytics.com/>
- [6] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 213–229.
- [7] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable Transformers for End-to-End Object Detection," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [8] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs Beat YOLOs on Real-time Object Detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16965–16974.
- [9] C. Fu, R. Liu, X. Fan, P. Chen, H. Fu, W. Yuan, M. Zhu, and Z. Luo, "Rethinking General Underwater Object Detection: Datasets, Challenges, and Solutions," *Neurocomputing*, vol. 517, pp. 243–256, 2023.
- [10] J. C. P. Cheng and M. Wang, "Automated Detection of Sewer Pipe Defects in Closed-Circuit Television Images Using Deep Learning Techniques," *Automation in Construction*, vol. 95, pp. 155–171, 2018.
- [11] H. Bharathwaj, "COROSSION DETECTION Dataset," Roboflow Universe, CC BY 4.0. [Online]. Available: <https://universe.roboflow.com/harish-bharathwaj/corossion-detection-6hmzy-fzart>